TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 7
Construir la aplicación

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 7

Qué vamos a ver

La unidad son dos clases. Esta es la primera: cómo se convierte el proyecto en un archivo instalable y firmado.

Hoy · Construir

- Qué entrega cada tienda y por qué va firmado
- EAS Build: perfiles, credenciales, distribución interna
- iOS sin tener una Mac
- Versionado de la app

La clase que viene · Publicar

- Íconos, splash y ficha de tienda
- Cuentas de desarrollador y EAS Submit
- Actualizaciones por aire con EAS Update
- Cómo se entrega el TP integrador

UNIDAD 7 · PARTE 1

Del proyecto al binario

Qué archivo pide cada tienda, y por qué ninguna acepta uno sin firmar.

Dónde estábamos

En la Unidad 1 quedó planteada la diferencia. Ahora la usamos.

DEBUG — LO QUE VENÍAMOS HACIENDO

- Metro sirve el JavaScript desde tu computadora.
- Recarga al guardar, herramientas de desarrollo.
- La app no funciona si apagás la máquina.

RELEASE — LO QUE SE PUBLICA

- El JavaScript va **adentro** del binario, como bytecode de Hermes.
- Minificado, optimizado, sin herramientas de desarrollo.
- Se instala y anda sola.

> Toda esta unidad trabaja sobre builds de **release**. Es la primera vez en el cuatrimestre que medimos la app como la va a ver el usuario.

Qué archivo pide cada tienda

.aab PLAY STORE

Android App Bundle. No se instala: es el paquete que Google recibe y del que genera un APK a medida de cada dispositivo e idioma.

.apk

El instalable de Android. Sirve para probar y para repartir a mano, **no** para subir a Play Store.

.ipa APP STORE

El equivalente de iOS. Solo se instala a través de la App Store o de TestFlight, nunca copiando el archivo.

> Google acepta **únicamente** `.aab` para apps nuevas. Un `.apk` subido a Play Console es rechazado.

Por qué todo va firmado

Ni Android ni iOS instalan una app sin firma digital. No es un trámite: es el mecanismo que sostiene las actualizaciones.

Qué garantiza

- **Identidad:** esta app la publicó quien dice publicarla.
- **Integridad:** el archivo no se modificó en el camino.
- **Continuidad:** la actualización viene del mismo autor que la versión instalada.

La consecuencia dura

Si la clave cambia, el sistema operativo considera que es **otra app**. No hay actualización posible: el usuario tendría que desinstalar y perder sus datos.
Por eso la clave se cuida más que el código.

Android: las dos claves

Google separa la clave con la que vos subís de la clave con la que el usuario recibe la app. Se llama **firma de apps de Play**.

Clave de carga

La que usás para firmar el `.aab` antes de subirlo. Google la usa para verificar que sos vos.

Si la perdés, se puede reemplazar pidiendo un restablecimiento.

Clave de firma de la app

Con esta Google firma los APK que llegan al dispositivo.

Nunca cambia durante toda la vida de la app.

Google la guarda por vos. **Es la que no se puede perder** — y por eso conviene no tenerla.

> Las claves viven en un **keystore** (`.jks`): un archivo binario con contraseña que funciona como repositorio de certificados.

iOS: certificados y perfiles

Apple usa dos piezas en lugar de una, y las dos las emite Apple.

Certificado de distribución

Identifica a la cuenta de desarrollador que firma. Equivale, en intención, al keystore de Android.

Perfil de aprovisionamiento

Dice qué app, firmada por quién, puede correr dónde. Para pruebas incluye la lista de dispositivos autorizados.

> Generarlos a mano es tedioso incluso para quien ya lo hizo, y clásicamente requería una Mac. Es exactamente el problema que resuelve la parte que sigue.

UNIDAD 7 · PARTE 2

EAS Build

Compilar en la nube, con las credenciales administradas.

Qué es EAS

Expo Application Services: los servicios en la nube que acompañan al SDK. Tres de ellos son esta unidad.

EAS Build

Compila la app en servidores de Expo y devuelve el `.aab`, `.apk` o `.ipa` firmado.

EAS Submit

Sube ese binario a Play Store o App Store desde la terminal.

EAS Update

Manda cambios de JavaScript a las apps ya instaladas, sin pasar por la tienda.

> Los tres funcionan con el **plan gratuito**. Los límites concretos los vemos en unas slides.

Los primeros tres comandos

```
# 1. La herramienta de línea de comandos de EAS
npm install --global eas-cli

# 2. Iniciar sesión con la cuenta de Expo (la misma de Expo Go)
eas login

# 3. Preparar el proyecto: crea eas.json
eas build:configure
```

> `eas build:configure` se corre **una vez por proyecto**. Lo único que hace es dejar un `eas.json` en la raíz, al lado de `package.json`.

eas.json y los perfiles

```json
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {}
  }
}
```

Un **perfil** es un conjunto de opciones con nombre. Estos tres vienen por defecto, pero los nombres son libres: podrías llamarlos como quieras.

> Sin `--profile`, EAS usa `production`.

Qué hace cada perfil

Perfil	Qué genera	Para qué
development	Build de debug con `expo-dev-client`	El development build de la Unidad 1. Nunca va a una tienda.
preview	`.apk` de release, sin herramientas de desarrollo	Que otra persona pruebe la app real. **Es el perfil del TP.**
production	`.aab` / `.ipa` firmados para la tienda	Publicar. No se instala directo en un dispositivo.

> `"distribution": "internal"` es lo que cambia el formato de Android de `.aab` a `.apk`. Sin eso, el archivo que baja no se puede instalar.

Las credenciales las genera EAS

```
eas build --platform android --profile production
```

La primera vez, la terminal pregunta qué hacer con la firma:

Generate new keystore

EAS crea el keystore, lo guarda cifrado en sus servidores y lo usa en cada build. No hay archivo que perder.

Usar uno propio

Si ya publicaste antes, subís tu keystore existente y EAS firma con ese.

> En iOS es el mismo mecanismo: iniciás sesión con tu cuenta de Apple Developer y EAS emite el certificado y el perfil por vos.

Qué pasa mientras compila

1 · Subida
EAS comprime el proyecto y lo manda. Lo que esté en `.gitignore` no viaja.

2 · Prebuild
Genera `android/` e `ios/` a partir de `app.json` y los config plugins.

3 · Compilación
Gradle o Xcode en un servidor limpio. El JavaScript se empaqueta a bytecode de Hermes.

4 · Firma
Se firma con las credenciales guardadas y queda el archivo para descargar.

> El comando espera hasta el final, pero se puede cortar sin problema: el build sigue en el servidor. `eas build:list` muestra el estado.

Distribución interna

Un build con `"distribution": "internal"` genera un **link** con código QR. Quien lo abre desde el teléfono, instala.

Android

Es un `.apk`. Se instala aceptando el aviso de "app de origen desconocido". Sin cuenta de desarrollador, sin tienda, sin revisión.

iOS CON FRICCIÓN

Aprovisionamiento ad hoc: solo instalan los dispositivos cuyo UDID estaba registrado **al momento de compilar**.
Se registran con `eas device:create`. Máximo 100 por año.

> En iOS, sumar un dispositivo nuevo obliga a **volver a compilar**. En Android el mismo `.apk` sirve para cualquiera.

Versionar la app

Hay dos números distintos y se confunden todo el tiempo.

`version` — la que se ve

La que figura en la ficha de la tienda: `"1.4.0"`. La elegís vos, cuando decidís que hay una versión nueva.

`versionCode` / `buildNumber`

El número interno de cada envío. Nadie lo ve. Tiene que subir en **cada** archivo que subas a la tienda.

> Olvidarse de incrementar el número interno es una de las causas más comunes de rechazo: la tienda ve un envío duplicado.

Que lo cuente EAS

```
{
  "cli": { "appVersionSource": "remote" },
  "build": {
    "production": { "autoIncrement": true }
  }
}
```

Con `appVersionSource: "remote"` el número interno lo guarda y lo incrementa el servidor de EAS en cada build. Los valores del `app.json` pasan a ignorarse.

> Es el comportamiento recomendado desde EAS CLI 12. Elimina de raíz el olvido de la slide anterior.

Los límites del plan gratuito

15 + 15	45 min	1	baja
builds por mes Android e iOS	tiempo máximo por build	build a la vez (concurrencia)	prioridad en la cola

Suficiente para el TP, pero no para compilar a cada rato. Cuando el cupo se agota o la cola está larga, queda el camino local de la Unidad 1: `eas build --local` o `npx expo run:`.

Enviar a las tiendas con **EAS Submit está incluido** en el plan gratuito, sin límite de envíos.

UNIDAD 7 · PARTE 3

iOS sin tener una Mac

Qué se puede hacer desde Windows o Linux, y qué no.

El mito y el dato

LO QUE SE CREE

- "Para hacer una app de iOS hace falta una Mac."
- Era cierto: compilar iOS requiere Xcode, que solo corre en macOS.

LO QUE PASA HOY

- La Mac la pone **EAS**: compila en sus servidores.
- Desde Windows o Linux salen builds de iOS reales, firmados.
- Lo que sigue haciendo falta es la **cuenta**, no la máquina.

> Cuenta de Apple Developer: **99 USD por año**. Es el límite real para un estudiante, y no lo resuelve ninguna herramienta.

Tres builds de iOS distintos

Destino	Necesita cuenta paga	Cómo se pide
Simulador de iOS solo corre en una Mac	No	`"ios": { "simulator": true }` en el perfil
iPhone real, interno	Sí	`eas device:create` + perfil `preview`
App Store / TestFlight	Sí	Perfil `production`

> El build de simulador es gratis pero no sirve de mucho sin una Mac donde abrirlo. La fila del medio es la que suele hacer falta.

Lo que EAS no resuelve

La revisión de Apple

Toda app y toda actualización pasa por revisión humana. Puede tardar días y puede ser rechazada.

El límite de 100

Cien dispositivos por año para distribución ad hoc. Borrar uno no libera el cupo hasta que se renueva la membresía.

Las reglas de la tienda

Permisos justificados, política de privacidad, contenido. El binario puede estar perfecto y el envío rechazarse igual.

> Android tiene su propia versión de esto, y es más dura de lo que parece. Lo vemos en la clase que viene.

UNIDAD 7 · ANEXO

Firma manual

El camino anterior a EAS. Conviene conocerlo, no usarlo.

Generar un keystore a mano

```
keytool -genkeypair -v \
  -keystore mi-app.jks \
  -alias mi-app \
  -keyalg RSA -keysize 2048 \
  -validity 10000
```

También sale desde Android Studio, con **Build > Generate Signed Bundle / APK**. Pide ubicación, contraseña, alias y años de validez.

> **Al menos 25 años de validez.** Si el certificado vence, no se puede firmar la actualización — y ya vimos qué significa eso.

Por qué no es el camino recomendado

MANUAL

- Un archivo crítico vive en una computadora.
- Si esa persona se va del equipo, el archivo se va con ella.
- La contraseña termina en un chat.
- Para iOS, además, hace falta una Mac.

ADMINISTRADO POR EAS

- Las credenciales viven cifradas en el servidor.
- Cualquier integrante autorizado compila.
- Se pueden descargar cuando quieras.
- Mismo flujo para Android y para iOS.

UNIDAD 7

Lo que sigue

- Íconos, splash y todo lo que la tienda mira antes que el código
- Las dos cuentas de desarrollador, y qué exige cada una
- : del binario a la ficha publicada
- : corregir un bug sin pasar por la tienda
- Cómo se entrega el TP integrador

Documentación: docs.expo.dev/build